

Recuperación

Introducción

A **Recuperación** é o proceso que permite devolver unha base de datos a un estado consistente despois de que ocorrera un fallo que a deixou nun estado inconsistente. Está baseada na **redundancia**, que permite copiar ou reconstruír os datos perdidos a partir de outros almacenados previamente.

Principais **mecanismos** utilizados para levala a cabo:

- Xestor de bases de datos transaccional.
- Ficheiros de log.
- Copias de seguridade (backups).
- Plan de salvaguarda.

Almacenamento e fallos

Tipos de almacenamento

Desde o punto de vista da recuperación, consideramos tres tipos de almacenamiento:

- **Almacenamento volátil:** O seu contido pérdese ante unha caída do sistema.
- **Almacenamento non volátil:** O seu contido non se perde salvo que se produza un fallo físico do dispositivo. Mantense ante unha caída do sistema.
- **Almacenamento estable:** Garante a conservación da información. Nunca se perde. É un tipo de almacenamiento teórico e utópico, que se pode tratar de simular con múltiples copias da mesma información.

Acceso a datos

Os datos gárdanse en disco (non volátil). A unidade de transferencia é o *bloque* (tamén chamado páxina).

Os bloques transfírense a un espazo en RAM (volátil) manexado polo SXBD: o *buffer cache*. Os programas acceden ós datos individuais utilizando a súa propia memoria.

Opcións implicadas:

- Transferencia entre buffer cache e disco:
 - Input(B): Le o bloque B de disco e transfíreo ao buffer cache.
 - Output(B): Escríbese o contido do bloque B no buffer cache en disco.

- Lectura e escritura de datos desde o programa:
 - Read(X, xi): Asigna á variable xi de programa o valor de X no buffer cache. Se X non está no buffer cache, antes transfírese desde disco mediante Input(B) (sendo B o bloque onde está a variable X).
 - Write(X, xi): Asigna á variable X no buffer cache o valor da variable de programa xi. Se X non está no buffer cache, antes debe recuperarse de disco mediante Input(B). A operación Write (X, xi) non chama a Output(B). Será invocada en algún momento do futuro, dependendo de distintas estratexias de escritura.

Fallos

Debido á súa natureza, poden ocorrer tipos de fallos:

- Erros nos datos, erros lóxicos do software que accede á BD, ou erros de concurrencia.
- Destrución non intencionada (descoidos, ...) ou intencionada (sabotaxe) de datos, ou incluso software e hardware asociado á BD.
- Caídas do sistema debido a fallos de software, hardware (incluído suministro eléctrico) que implican a perda dos datos en memoria.
- Fallos de dispositivo (principalmente discos).
- Catástrofes naturais ou provocadas (terremotos, incendios, ...).

Desde o punto de vista da recuperación, podemos clasificar estes fallos en:

- **Fallos sen perda de almacenamento:** Non se perden datos, pero a BD pode non ser consistente.
- **Fallos con perda de almacenamento volátil:** Existen os ficheiros da BD, pero pida que non sexan consistentes. Despois de recuperar o sistema debe restaurarse a BD a un estado correcto. Normalmente implica a utilización de ficheiros de log.
- **Fallos con perda de almacenamento non volátil:** Os ficheiros de BD desapareceron ou son incorrectos. Require a utilización dun backup para restaurar os datos, e habitualmente un proceso posterior, usando os ficheiros de log, para deixar a BD nun estado consistente.

Transaccións

Definición

Unha **transacción** é unha unidade lóxica de execución, formada por unha secuencia de unha ou máis operacións de lectura e/ou escritura sobre a base de datos.

Unha transacción pode rematar de 2 formas:

- Confirmando os cambios (**COMMIT**).
- Anulando os cambios (**ROLLBACK**).

Propiedades **ACID** (Atomicity, Consistency, Isolation, Durability) dunha transacción:

- **Atomicidade:** Execútanse todas as operacións, ou non se executa ningunha.
- **Consistencia:** A transacción leva a BD dun estado consistente a outro estado consistente.
- **Illamamento:** As transaccións execútanse de forma independente unhas de outras. Os cambios parciais dunha transacción non deberían ser visibles desde outras.
- **Durabilidade/Persistencia:** Os cambios realizados por unha transacción confirmada quedarán gardadas na BD e non se perderán incluso na presenza de fallos.

As transaccións son a unidade fundamental para garantir o control de concurrencia e a recuperación ante fallos.

Estados dunha transacción

Unha transacción está **activa** durante a execución das operacións.

Desde este estado activa, se solicita un **COMMIT**, a transacción pasa a estar **parcialmente confirmada**.

Tras unha serie de comprobacións e accións a levar a cabo durante o proceso de commit, se todo ten éxito, pasa a estar **confirmada**. Debe garantirse que os cambios quedan almacenados de forma persistente na BD.

Estando activa, se se solicita un **ROLLBACK**, paso ao estado de **fracasada**. Tras as accións necesarias (desfacer cambios, se é necesario) pasa a **abortada**.

Se as comprobacións a levar a cabo no estado de parcialmente confirmada non teñen éxito (por exemplo, fallo na comprobación dunha restrición aprazada ou fallos de concurrencia), pasará a fracasada e finalmente a abortada.

Recuperación baseada en logs

Ficheiros de log

Os **ficheiros de log**, ou **redo logs** son ficheiros de pequeno tamaño que almacenan cambios realizados na base de datos.

Os ficheiros de log conteñen **rexistros de transacción**, que almacenan:

- O identificador da transacción.
- O tipo de rexistro, que pode ser:
 - Inicio de transacción: **START**.
 - Finalización da transacción: **COMMIT** ou **ROLLBACK**.
 - Modificación de datos (DML): **INSERT, DELETE, UPDATE**.
- Só para os rexistros de modiciación:
 - Identificador do dato.

- Imaxe anterior: o valor do dato antes do cambio (só para `DELETE` e `UPDATE`; só para actualizacións inmediatas).
- Imaxe posterior: o valor do dato despois do cambio (só para `INSERT` e `UPDATE`).

Existen tamén **rexistros de checkpoint** ou puntos de verificación, que describimos máis adiante.

Note

Por simplicidade, os exemplos de modificacións de datos deste documento serán update, e os rexistros de log terán un aspecto similar a: <T1, DatoA, imaxeAnterior, imagePosterior>

Técnicas de actualización

Os ficheiros de log utilízanse con distintas técnicas de actualización:

- **Actualizacións inmediatas:** Cando se solicita a modificación dun dato:
 1. Escríbese o rexistro corresponde no log.
 2. Modifícase o dato nos buffers de datos da BD.

Isto implica que se ocorre un fallo antes de confirmar a transacción, esta puido ter feitas modificacións que hai que descafer.

Ademais, a modificación nos buffers de datos puido terse trasladado a disco (*output(B)*) ou non.

- **Actualizacións diferidas:** Cando se solicita a modificación dun dato:

Escribese o rexistro de log, pero *non* se modifican os datos. A modificación dos datos faise no momento do `commit` da transacción.

Neste caso, a transacción nunca ten cambios parciais dunha transacción sen confirmar, polo que nunha hai que desfacer cambios.

Operacións de recuperación

En caso de erro, cando o sistema se recupera examina o ficheiro de log e decide que facer coas transaccións. Utilízanse 2 operacións: *redo* e *undo*. Ambas operacións afectan a todas as modificacións de datos que realiza a transacción.

redo(Ti): Refai os cambios da transacción T_i , asignando a cada dato modificado a imaxe posterior almacenada no log. Tamén se coñece como *roll forward* ou recuperación en avance.

undo(Ti): Desfai os cambios da transacción T_i , en orden inverso ao realizado na transacción.

Tamén se denomina recuperación cara atrás. Consiste en asignar a cada dato modificado a imaxe anterior almacenada no rexistro de log correspondente.

Ambas operacións son idempotentes:

```
redo(redo(...redo(Ti))) = redo(Ti)
undo(undo(...undo(Ti))) = undo(Ti)
```

Isto é importante porque:

- Permite refacer ou desfacer unha transacción sen saber se (parte das) modificacións foron ou non levadas a disco.
- Se o sistema sofre unha caída durante o proceso de recuperación, este proceso pode repetirse sen causar problemas.

Actualizacións inmediatas

Recordemos que son actualizacións inmediatas, ao producirse unha modificación, gárdase un rexistro de log e modifícanse os buffers da BD. Non sabemos cando se trasladan a disco estes cambios.

Cando se produce o fallo (consideramos só perda de almacenamiento volátil, sen fallos de dispositivo), cada transacción T_i puido:

- Rematar sen confirmar. Neste caso, dado que as actualizacións inmediatas modifican o buffer da BD, debemos desfacer eses cambios mediante $undo(T_i)$, desfacendo as operacións seguindo a orde inversa de como están no log, indo "cara atrás".
- Rematar confirmándose. Neste caso, hai que realizar $redo(T_i)$ seguindo a orde "cara adiante" das operacións no log. Basicamente, necesitamos facer *redo* porque non sabemos se as modificacións se levaron ou non a disco.

Important

As operacións *undo* deben facerse antes que as *redo*.

Dado que requirimos operacións *undo* e *redo*, necesitamos que o ficheiro de log almacene para cada modificación a imaxe anterior e posterior do dato.

// Poner foto de la diapositiva 17

Actualizacións diferidas

Recordemos que son actualizacións inmediatas, ao producirse unha modificación, gárdase un rexistro de log e modifícanse os buffers da BD. Non sabemos cando se trasladan a disco estes

cambios.

Cando se produce o fallo (consideramos só perda de almacenamiento volátil, sen fallos de dispositivo), cada transacción T_i puido:

- Rematar sen confirmar. Neste caso, dado que os buffers de datos non se modificaron, non hai que facer nada con estas transaccións.
- Rematar confirmándose. Neste caso, hai que realizar $redo(T_i)$ seguindo a orde "cara adiante" das operacións no log.

Dado que requerimos operacións *redo* e non *undo*, neste caso non necesitamos que o ficheiro de log almacene para cada modificación a imaxe anterior do dato. Basta coa imaxe posterior para o *redo*.

// Poner foto de la diapositiva 19

Check points

Un **checkpoint** ou **punto de verificación** forza a gravar os cambios dos buffers da BD en disco.

Están planificados para realizarse de forma periódica, con obxectivos de:

- Reducir o tempo de recuperación (utilizando só unha parte dos ficheiros de log).
- Liberar (para a súa reutilización) os ficheiros de log.

Un checkpoint leva a cabo as seguintes accións:

1. Suspende temporalmente a execución de transaccións en curso.
2. Escribir nos ficheiros de log os buffers de log modificados.
3. Escribir na BD en disco os bloques de datos (buffer cache) modificados.
4. Escribir no log (en disco) un rexistro **CHECKPOINT** que inclúa a lista de transaccións *activas* (iniciadas, pero que non foron anuladas nin confirmadas).
5. Restaurar a execución das transaccións.

Recuperación (tras un fallo sen perda do almacenamento non volátil):

O uso de checkpoints asegura que para a recuperación as transaccións rematadas *antes* do checkpoint poden ser ignoradas:

- As transaccións confirmadas (tanto actualizacións diferidas como inmediatas) modificaron os buffers de datos e estes cambios foron escritos en disco.
- As transaccións anuladas restauraron os valores nos buffers de datos (actualizacións inmediatas) ou nunca fixeron os cambios (actualizacións diferidas), e os buffers foron escritos en disco.

Polo tanto, para unha recuperación deste tipo só se necesita considerar:

- As transaccións activas (que están no rexistro de **CHECKPOINT**)
- As transaccións iniciadas despois do checkpoint.

Buffers de logs e buffers de datos

Existe unha zona de memoria para almacenar os **buffers de logs**.

O protocolo WAL (**Write-Ahead Logging**, protocolo de escrituras anticipadas) especifica que sempre debe rexistrarse o cambio nos logs antes que na base de datos física. Isto inclúe:

- Moficiar o buffer de log antes que o buffer de datos.
- Cando se volca un buffer de datos a disco, antes deben volcarse os rexistros do buffer de log relacionados ao ficheiro de log.

O obxectivo é asegurarse de grabar a información de como refacer/desfacer as transaccións antes de facer os cambios. Se se volcasen a disco os buffers de datos antes, e houbera un fallo antes de volcar a disco o log, non seríamos capaces de refacer/desfacer as transaccións.

Proceso de confirmación:

Cando se solicita un **COMMIT** dunha transacción, debe volcarse os buffers de log desa transacción (rexistro **START**, todos os rexistros de modificacións, e o rexistro de **COMMIT**) ao ficheiro de log antes de completar a confirmación da transacción.

Estratexias de volcado de buffers de datos:

- **Roubar**: Unha páxina no buffer de datos pode volcarse a disco antes de confirmar a transacción, ou **Non roubar**: non pode volcarse antes de confirmar.
- **Forzar**: Todas as páxinas modificadas no buffer de datos deben volcarse a disco cando se confirma a transacción, ou **Non forzar**: non se obriga a volcar a disco ao confirmar.

O habitual, por eficiencia, é Roubar e Non forzar.

Logs online e offline

Normalmente unha BD utiliza varios ficheiros de físicos para implementar o "log da base de datos".

Reciben o nome xenérico de **logs online**.

- Teñen un tamaño fixo.
- O mínimo son 2 ficheiros físicos.
- Estarán almacenados en soportes separados dos ficheiros de datos da BD.
- Poden estar multiplexados (existir varias copias dos ficheiros de log).

Estes ficheiros utilízanse de forma cíclica, sobreescribindo os ficheiros. Isto implica que:

- O tamaño de log usable é limitado.
- Hai unha "ventá" de recuperación (transaccións dispoñible no log) tamén limitada.
- O uso de checkpoints facilita que se liberen ficheiros de log (aqueles que non sexan necesarios para a recuperación sen perda de almacenamento non volátil).

Para incrementar esta ventá de recuperación, ou para unha recuperación con perda de almacenamento volátil (probablemente necesite un fragmento grande do log), os xestores implementan habitualmente o **arquivado de ficheiros de log**.

Cando se emplea un ficheiro de log, pásase a usar, sobreescribindo, o seguinte ficheiro. O ficheiro que se completou cópiase (arquívase) nunha ubicación diferente, utilizando un número de secuencia. Entes logs denomínanse normalmente **logs offline**.

- Nunca se sobreescriben.
- O número de secuencia permite utilízalos de forma ordenada no proceso de recuperación.
- Permiten unha ventá de recuperación máis ampla.
- Poden gravarse en medios de almacenamiento de só lectura, ou secuenciais.

Recuperación dun fallo con perda de almacenamento non volátil

As técnicas baseadas en logs vistas ata agora partían dunha BD física que podía non ser consistente, pero non estaba danda.

Esta BD física non será usable tras un fallo físico do dispositivo, ou a corrupción do sistema de ficheiros onde residen os ficheiros de datos.

Para realizar a recuperación, necesitamos ter un *backup* ou copia de seguridade da base de datos. Neste caso, o proceso de recuperación sería:

1. Restaurar os ficheiros de datos.
2. Realizar a recuperación baseada en logs.
 - Utilizarase a técnica apropiada (actualizacións inmediatas ou diferidas).
 - Utilizará os logs online e probablemente os logs offline (que quizás sexa necesario restaurar desde cinta).

É posible tamén que o fallo implique a perda dos ficheiros de configuración e incluso o software do SXBD.

Neste caso, o proceso de recuperación implica a restauración ou reinstalación do software, aplicación da configuración, e finalmente executar os 2 pasos anteriores.

Copias de seguridade

Que resgardar

Unha **copia de seguridade** ou **backup** é unha copia de certos datos, que se almacenan normalmente nun soporte distinto, que pode ser utilizada para restaurar os datos orixinais despois da súa perda.

Es previsión dun fallo catastrófico que faga que se perda toda a información, debería facerse backup de:

- Software: como mínimo o SXBD.
- Ficheiros de configuración e outros ficheiros auxiliares do SXBD.
- Ficheiros de datos da BD.
- Logs offline

A copia de seguridade do software realizarase habitualmente utilizando ferramentas de backup do sistema. Pode tamén considerarse gardar os medios de instalación orixinais do SXBD.

Os ficheiros de datos e logs offline poderán resgardarse usando ferramentas do sistema operativo, ou ferramentas do propio SXBD. Será normalmente unha copia física dos ficheiros, polo que a BD pode estar nun estado inconsistente (no momento de fallo podía haber transaccións activas e/ou buffers en memoria con cambios non trasladados a disco).

Os logs online, pola súa característica extremadamente dinámica, non son resgardados habitualmente nas copias de seguridade da BD. En previsión de que sexan necesarios, están habitualmente multiplexados (varias copias de cada ficheiro de log).

Proceso de recuperación

A recuperación da BD dependerá do tipo de fallo sufrido.

- Fallo sen perda de almacenamento non volátil: Non será necesario utilizar as copias de seguridade, basta con aplicar os logs online como se describe na sección anterior.
- Fallo con perda de almacenamento non volátil: Require o uso de backups. Os pasos a levar a cabo serán:
 1. Se o fallo foi catastrófico, pode perderse tamén o propio software. Deben restaurarse ou reinstalarse o Sistema Operativo e o SXBD, e restaurar a configuración e ficheiros auxiliares do SXBD.
 2. Restaurar os ficheiros de datos a partir do último backup.
 3. Utilizando os offline logs, e a estratexia axeitada (actualizacións inmediatas / diferidas), devolver a BD ao último estado consistente dispoñible nestes logs.
 4. No caso de que non se perderan os online logs, aplicalos para obter o último estado consistente da BD.

Note

Existe un tipo de backup (backup en frío) que se fai co SXBD parado. Non hai transaccións activas e todos os buffers foron previamente volcados a disco, polo que permite a recuperación ata o instante do backup, sen usar os logs

Exportación de datos vs. Backups físicos

Ferramentas de exportación

Ferramentas como `pg_dump` / `pg_dumpall` (PostgreSQL), `mysqldump` (MySQL/MariaBD) ou `exp` / `expdp` (Oracle) permiten *exportar* os datos a ficheiros.

Estes datos poderán logo ser importados (con `pg_resotre` / `psql`, `mysql` ou `imp` / `impdp`).

A veces estes ficheiros denomínanse *backups lóxicos*.

A exportación fai copia dunha instantánea dos datos vistos por un programa que accede nun entorno concurrente á base de datos.

Polo tanto:

- Non verán os datos que están sen confirmar por outras transaccións.
- Non son utilizables en combinación cos ficheiros de log.

O único que permiten é restaurar a BD ao estado existente no instante da realización da copia. Require unha instalación operativa do SXBD.

Backups físicos

Son unha copia física dos ficheiros da BD.

Poden reutilizarse desde o sistema operativo, ou usando ferramentas específicas.

Poden usarse para resgardar os logs offline.

Poden usarse na recuperación combinando a restaruación dos ficheiros de datos coa aplicación dos logs.

Backups periódicos

Os buckups de datos realizanse normalmente de forma periódica.

En caso contrario: A recuperación podría ser extremadamente lenta. Debería restaurar ese backup e logo aplicar os logs offline para refacer/desfacer todas as transaccións desde ese backup inicial.

Se o xestor o permite, podemos planificar 2 tipos de backups periodicamente:

- **Backup completo:**
 - Fai unha copia completa dos ficheiros de datos.
 - Se a BD é grande, este tipo de backup planifícase habitualmente a intervalos non demasiado cortos.
- **Backup incremental:**
 - Fai copia dos bloques de datos que cambiaron desde o último backup.
 - Algunhas variantes copian os bloques cambiados desde o último backups completo, outras os cambiados desde o último backup de calquera tipo.
 - Normalmente ocupan moito menos espacio que os completo polo que poden planificarse máis frecuentemente

Ambos poderían, ademais, incluír ou non logs offline.

Para restaurar os ficheiros da BD, recupérase o último backup completo, e logo aplicárase por encima ("parcheando") o último backup incremental.

Plan de salvaguarda

O plan, ou política, de salvaguarda total, permitirá a xestión do risco que supoñería a perda da BD.

Se usamos a norma ISO/IEC 27001, co ciclo de Deming ou PDCA, teríamos algo similar a:

- **Plan:** Especificar que información debe gardarse, con que técnicas, e con que periodicidade. Tamén se especificará como se verificarán eses backups.
- **Do:** Implementar os backups se realizan e probar periodicamente a restauración da BD noutro entorno.
- **Check:** Verificar que os backups se realizan e probar periodicamente a restauración da BD noutro entorno.
- **Act:** Comprobar, dependendo do verificado, se hai que axustar algunha acción.

Exemplo de salvaguarda

- Backup do software: único (tras a instalación).
- Backup da configuración e ficheiros auxiliares: diario.
- Backup de datos:
 - Completo mensual.
 - Incremental diario.
 - Ambos inclúen os logs offline.
- Probas:
 - Verificación diaria (automática) da realización dos backups.

- Proba de recuperación: en instalación de probas, 2 veces ao ano.